System Design Documentation for LibraQ

Version 2.0

Prepared by: Taisia, Timothy, Akhanali, Parnia

November 16th 2025

Table of Contents:

CRC cards..... 3
Description of system interaction with the environment..... 6
Operating Environment:..... 6
Data Storage:..... 6
External Dependencies..... 6
Assumptions and constraints:..... 6
Architecture of the system..... 7
Dealing with errors..... 7
Input and User Errors:..... 7
Business Logic Errors..... 7
File Storage and Data Persistence Errors..... 7

CRC cards

Enum:

Class Name: Genre	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • List different Genres and their associated display name.	**Collaborators:** • None

Entities:

Class Name: User (abstract)	
Parent Class (if any): None **Subclasses (if any):** Librarian, Renter	
Responsibilities: • Store user information (username, password, name, email).	**Collaborators:** • None

Class Name: Librarian	
Parent Class (if any): User **Subclasses (if any):** None	
Responsibilities: • Store attributes and permissions specific to librarians.	**Collaborators:** • None

Class Name: Renter	
Parent Class (if any): User **Subclasses (if any):** None	
Responsibilities: • Store attributes and permissions specific to Renters	**Collaborators:** • None

Class Name: Book

Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Store all book attributes (title, author, ISBN, genre, availability).	**Collaborators:** • Genre

Class Name: CheckoutReceipt	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Store all attributes corresponding to a checkout receipt, including the user and book, the date the book was checked out, the due date, date book was returned, and fine amount associated with that checkout.	**Collaborators:** • User • Book

Repositories:

Class Name: BookRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of Book objects. • Provide methods for finding a book based on specific attributes. • Provide methods for adding/removing books.	**Collaborators:** • Book

Class Name: UserRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of User objects. • Provide methods for finding a	**Collaborators:** • User

user based on specific attributes. • Provide a method for adding Users.	

Class Name: CheckoutReceiptRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of CheckoutReceipt objects. • Provide methods for finding a CheckoutReceipt based on User or Book. • Provide a method for adding a new receipt to DB.	**Collaborators:** • CheckoutReceipt • User • Book

Controllers:

Class Name: BookController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Loading the books detail page for a specific book • Calling CheckoutService when a user clicks on the checkout button on the page	**Collaborators:** • BookService • CheckoutService • UserService

Class Name: HomeController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Responsible for loading the home page using appropriate services (loading the book list using BookService). • Routing clicks on books to the appropriate book details page.	**Collaborators:** • BookService

Class Name: RegistrationController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Handle the User registration form submission ● Validates User;s input data(email format, password strength and match) ● Redirects to Home page on successful registration	**Collaborators:** ● UserService ● Renter

Class Name: AuthenticationController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Display the log in form page	**Collaborators:** ● SecurityConfig

Class Name: UserDetailsController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Gets currently logged in User's data ● Gets checkout receipts for currently logged in user	**Collaborators:** ● User ● CheckoutReceipt

Services:

Class Name: BookService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Handle search queries of books. ● Provide methods for	**Collaborators:** ● Book ● BookRepository

adding/removing books.	

Class Name: CheckoutService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Manage book checkouts and checkout extensions for renters. ● Handle check-ins when librarians process returns. ● Update availability status of Books. ● Manage reservations of books by renters.	**Collaborators:** ● BookRepository ● CheckoutReceiptRepository ● User, Renter ● Book

Class Name: UserService	
Parent Class (if any): UserDetailsService **Subclasses (if any):** None	
Responsibilities: ● Provides User CRUD operations ● Implements UserDetailsService for authentication ● Loads User by email for login ● Determines User's role for authorization	**Collaborators:** ● User ● UserRepository ● PasswordEncoder

Configuration:

Class Name: SecurityConfig	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Configures the authentication and authorization systems ● Configures the log in processes ● Configures successful log in redirect ● Configures the log out logic ● Configures public and protected	**Collaborators:** ● SpringSecurity ● UserService ● AuthenticationController

URL's	

Description of system interaction with the environment

Operating Environment:

- **Operating System:** Cross-platform (Windows, macOS, or Linux)
- **Development Environment:** Eclipse IDE for Enterprise Java Developers
- **Programming Language:** Java 17
- **Framework:** Spring Boot (version 3.x)
- **Build Tool:** Maven
- **Browser Support:** Any modern web browser (e.g., Chrome, Firefox, Edge) can be used to access the web interface.

Data Storage:

- Data is stored using Springboot H2 Database.
- In the file system it is actually stored in data/librarydb.mv.db file

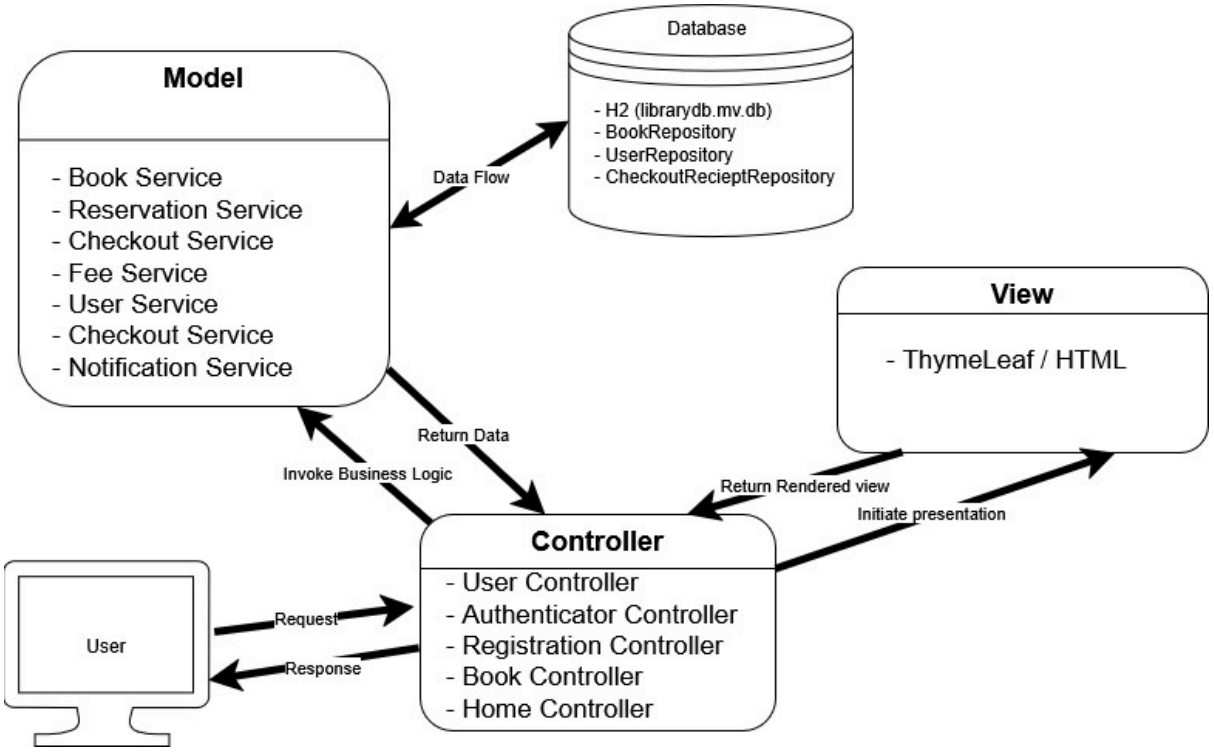
External Dependencies

- Thymeleaf – for rendering server-side HTML templates that display dynamic content.
- H2 Database
- Spring Security

Assumptions and constraints:

- The system assumes a single active local deployment (no distributed or concurrent multi-user setup).
- Network connectivity is not required, since the application runs fully offline on localhost.

Architecture of the system



Dealing with errors

Input and User Errors:

Examples: Invalid login credentials.
Handling:

- Inputs are validated and Controllers catch exceptions such as InvalidInputException or BookNotFoundException and return messages that will be displayed to the user (e.g., “Book not available”, “Invalid credentials”).

Business Logic Errors

Examples: Trying to check out a book that’s already checked out, or reserving a book twice.
Handling:

- CheckoutService and ReservationService validate all operations before performing updates.
- Violations throw custom exceptions, caught by controllers and shown as alerts.